

1.5+

1. 2.

min-max
power of an element

20 day

Lec_33

3. Binary Search

searching

$n = 50$

$O(n)$
 $O(n)$
 $O(n)$

array = [10, 5, 4, 1, 6, 9, 28, 31, 3]

element x
 $x = 20$

```
for (i = 0; i < n; i++)  
{  
    if (array[i] == x)  
        return i;  
}
```

Linear Search

memory address not
 $O(n)$ (WR)

return -1

(LS)
(BS)

Linear search $TC = O(n)$
Binary search $TC = O(\log n)$

$$\log n \ll n$$

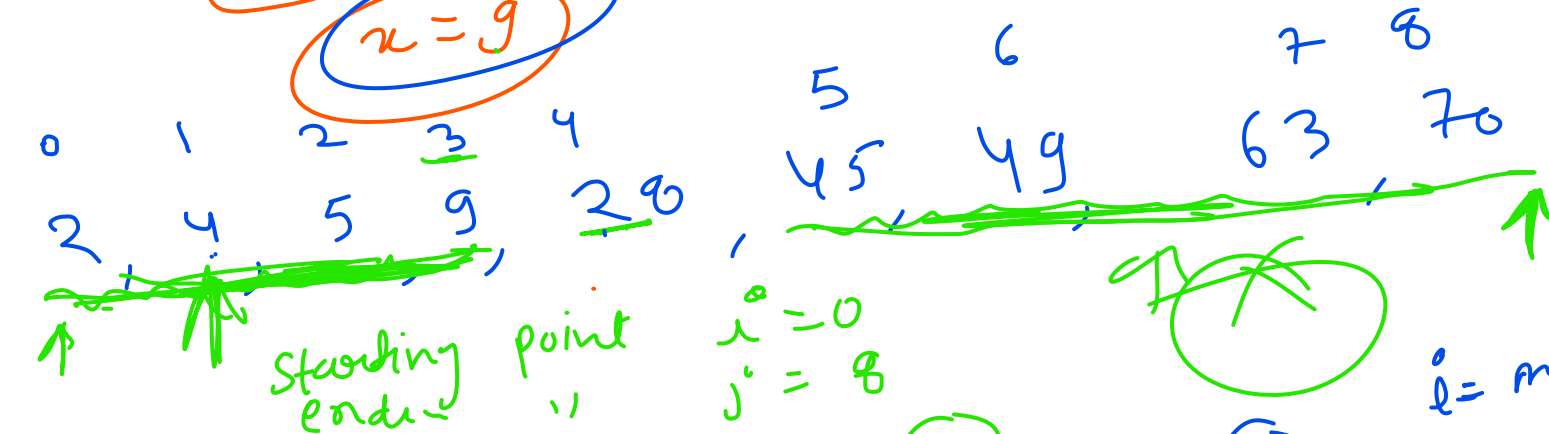
Input: Sorted Array of n elements , an element x

Output: Location of Element x , if exist, if not return -1

Example:

array $\Rightarrow a$ [2, 4, 5, 9, 20, 45, 49, 63, 70]

index
elements



① $mid = \frac{(l+r)}{2} = \frac{0+8}{2} = 4$
 $a[4] = 20$
 $x < a[4]$
 $x = 9$

③ $a[3] = 9$
 $x = 9$

$l = mid + 1 = 2$
 $j = 3$
 $mid = \frac{2+3}{2} = \frac{5}{2} = 2$
 $a[2] = 5$
 $l = 3, j = 3/2 = 1$
③

(2)

Starting point, $l=0$
end = $4 \Rightarrow \text{mid} - 1 \Rightarrow 4 - 1 = 3$
 $\text{mid} = \frac{l+r}{2} = \frac{0+3}{2} = 1.5 \Rightarrow 1$ $\text{ans} = 4$

Approach/Algo:

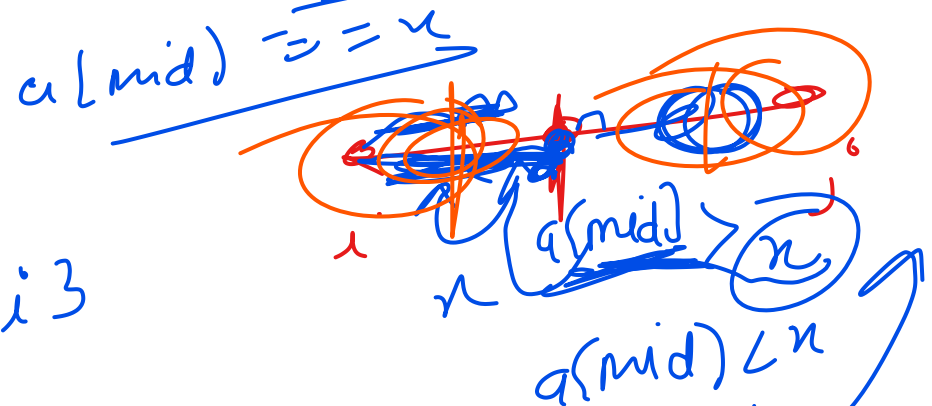
$a =$
 $n = 9$

~~2, 4, 5, 9, 26, 45, 49, 63, 70~~

Algorithm With Recursion:

$[2, 4, 3, 9, 11]$
 $n = 5$

$i = 0, j = 4$
 $mid = \frac{i+j}{2} = \frac{0+4}{2} = 2$
 $a[mid] = a[2] = 3$



return

$T(c) = 1$
 $T(n) = c + T(n/2)$

$BS(a, i, j, n)$

if ($i == j$)

{ if ($a[i] == n$) { return i }
 else return -1;

else

{ $mid = \frac{i+j}{2}$

if ($a[mid] == n$) { return mid };

if ($a[mid] > n$)

return $BS(a, i, mid-1, n)$;

if else ($a[mid] < x$)
return BS(a, mid+1, s, x)

Time complexity

$$T(n) = T\left(\frac{n}{2}\right) + C$$

Time Space Complexity Analysis:

$$T(1) = 1. \quad T(n) = T\left(\frac{n}{2}\right) + C$$

$$f(n) \sim n^{\log_b a}$$

$$C \mid n^{\log_2 1}$$

$$C \mid n^0$$

equal mean (all)

$$\rightarrow \frac{C \times \log n}{O(\log n)}$$

$$\text{space complexity} = \underline{O(\log n)}$$

$$\begin{aligned} T(1) &= O(1) \\ S(1) &= O(\log n) \end{aligned}$$

Algo Without Recursion:

BS(a, i, j, n)

```
while(i <= j)
{
    if(i == j)
    {
        if(a[i] == x)
            return i;
        else
            return -1;
    }
}
```

mid = (i + j) / 2
if (a[mid] == x)

if (a[mid] > x)

j = mid - 1;

else i = mid + 1;

return mid;

Recursion can be converted into loop.



TC = O(log n)

Linear vs Binary: TC and SC Comparisons:

Linear

$$TC, BC \Rightarrow O(1)$$

$$TC, WC \Rightarrow O(n)$$

Binary search

$$O(1)$$

$$O(\log n)$$

Space complexity!



Linear = $O(1)$

Binary = $O(\log n)$

Stack needed for
the recursion



Previous

Bitum

70-90

90x20

33

~~60~~
60

50

20

no